

Wapiti — Study Guide

Wapiti is an open-source **web application vulnerability scanner** that works differently from Nikto/Nuclei — instead of checking for known files/CVEs, it **crawls a site and actively injects payloads** into every form, parameter, and input it finds, functioning as a lightweight, automated black-box application security tester.

1. What Makes Wapiti Different From Nikto/Nuclei

	Nikto	Nuclei	Wapiti
Approach	Checks known files/paths/versions	Runs template-defined checks	Crawls + actively fuzzes every input
Finds	Known misconfigs/outdated software	Whatever templates cover	Actual injection vulnerabilities (SQLi, XSS, etc.) discovered by testing
Requires crawling	No	No	Yes — builds a site map first
Best for	Fast server-level triage	Broad, fast CVE/exposure sweeps	Application-layer logic flaws

Wapiti is closer in spirit to a lightweight, automated version of what you'd do manually in Burp Suite/OWASP ZAP — it's a **black-box, dynamic application security testing (DAST)** tool, not a signature/version checker.

2. Installation

```
pip install wapiti3
# or on Kali - pre-installed / apt available
sudo apt install wapiti
```

3. Basic Usage

```
wapiti -u http://target.com
```

Wapiti's process: **crawl the site** (discover pages/forms/parameters) → **attack** (inject payloads into everything found) → **report**.

4. Key Flags

Flag	Meaning
<code>-u</code>	Target URL
<code>-m</code>	Modules to run (specific vuln classes)
<code>-d</code>	Max crawl depth
<code>--scope</code>	Crawl scope (page, folder, domain)
<code>-a</code>	Authentication (username:password)
<code>--cookie</code>	Provide session cookies
<code>-p</code>	Proxy (route through Burp/ZAP for visibility)
<code>-o</code>	Output directory/file
<code>-f</code>	Report format (html, json, txt, xml)
<code>-v</code>	Verbosity level

5. Modules — Choosing What to Test For

```
wapiti -u target -m sql,xss,exec
```

Module	Tests For
<code>sql</code>	SQL injection
<code>xss</code>	Cross-site scripting
<code>exec</code>	Command execution
<code>file</code>	File inclusion (LFI/RFI)
<code>csrf</code>	Cross-site request forgery
<code>ssrf</code>	Server-side request forgery
<code>crlf</code>	CRLF injection
<code>htaccess</code>	.htaccess misconfiguration
<code>backup</code>	Backup file discovery
<code>wp_enum</code>	WordPress-specific enumeration
<code>shellshock</code>	Legacy Bash Shellshock vuln

```
wapiti -u target -m sql,xss,csrf,ssrf,file -o report_dir -f html
```

6. Handling Authentication

Since Wapiti crawls the whole site, testing anything behind a login requires providing credentials/session state:

```
wapiti -u target -a "username%password" # basic form login
wapiti -u target --cookie cookies.txt # or provide p
re-authenticated cookies
```

Getting authenticated crawling right is often the trickiest part of running Wapiti effectively — many real vulnerabilities live behind login, and an unauthenticated scan only tests the public-facing surface.

7. Practical Example — Full Scan Against DVWA

```
wapiti -u http://dvwa.local/vulnerabilities/sqli/ -m sql -a
admin%password --cookie cookies.txt -o wapiti_dvwa -f html
```

Since you've already worked with DVWA in your earlier study, running Wapiti against it and comparing its automated findings to the vulnerabilities you already know exist (by DVWA's design) is an excellent calibration exercise — you'll learn exactly what Wapiti catches automatically vs. what still needs manual testing.

8. Reading Wapiti Output

```
[+] SQL Injection in http://target/page.php?id=1
    Evil value: 1' OR '1'='1
    HTTP GET
    ---
    Confirmed vulnerability
```

Wapiti attempts to **confirm** findings by actually triggering the vulnerable behavior (e.g., detecting a SQL error response or successful injection

response), which generally gives it a lower false-positive rate than pure signature-matching tools like Nikto — though manual verification is still always the professional standard before reporting.

9. Wapiti's Limitations

- **Struggles with heavy JavaScript/SPA applications** — its crawler is not a full browser engine, so it can miss content rendered dynamically (React/Angular/Vue apps) — this is exactly where OWASP ZAP's browser-based crawling has an edge
- **Can be slow on large sites** — active injection testing against every discovered parameter takes time
- **Less actively maintained/smaller community than ZAP/Burp** — fewer plugins/extensions, though the core injection-testing functionality remains solid

10. Where Wapiti Fits in a Web App Testing Workflow

```
Nikto/Nuclei          → fast triage: known files, misconfigs, CVEs
↓
Wapiti                → automated black-box injection testing across the whole crawled site
↓
Manual testing(Burp/ZAP)→ deep-dive on anything Wapiti flagged, plus business-logic
                        flaws no automated tool can find
```

Wapiti fills the gap between "fast automated checks" and "fully manual testing" — it automates the *injection testing* labor (trying hundreds of payload variations across every input) that would take significant manual time in Burp, while still requiring human judgment to interpret and confirm results.

11. Kill Chain / ATT&CK Mapping

Framework	Mapping
Cyber Kill Chain	Reconnaissance → Weaponization (confirmed injection points feed directly into exploit development)
MITRE ATT&CK	T1190 (Exploit Public-Facing Application) — Wapiti's confirmed findings are direct T1190 candidates

Quick Reference Cheat Sheet

```
wapiti -u http://target.com # basic scan
wapiti -u target -m sql,xss,csrf # specific
modules
wapiti -u target -a user%pass --cookie c.txt # authent
icated scan
wapiti -u target -o report -f html # HTML r
eport
wapiti -u target -p http://127.0.0.1:8080 # route thr
ough Burp for visibility
```